

DSA0111 – ASSIGNMENT

BANK ACCOUNT MANAGEMENT SYSTEM USING OBJECT-ORIENTED PROGRAMMING IN C++

1. TITLE & INTRODUCTION

Title: "Bank Account Management System Using Object-Oriented Programming in C++"

In the modern software industry, applications that handle financial data must be secure, modular and easy to maintain. A Bank Account Management System is a classic real-world problem that captures the essence of Object-Oriented Programming (OOP): it involves entities with private data (account balances), behaviours that differ by account type (interest calculation), and a natural hierarchy (Savings and Current accounts are both types of Bank Accounts).

This assignment implements such a system in C++ to demonstrate, in a single working program, the core pillars of OOP — classes and objects, encapsulation, access specifiers, inheritance, function overloading, function overriding, virtual functions/runtime polymorphism, data security, and code reusability. The system models a base BankAccount class and two specialised account types, SavingsAccount and CurrentAccount, each with its own interest and withdrawal rules.

2. PROBLEM STATEMENT

A software development organisation needs a C++ application to manage bank accounts belonging to different customers. Each account must store an account number, the account holder's name, and a balance, and it must support depositing and withdrawing money. However, different types of accounts behave differently: a Savings Account earns interest on its balance, while a Current Account allows the holder to withdraw beyond the available balance up to a fixed overdraft limit but does not earn any interest.

The system must guarantee data security by preventing balances from being changed directly from outside a class, must avoid duplicating common logic between account types, and must allow the interest calculation to be selected automatically at runtime based on the actual type of account being processed — even when all accounts are accessed through a single base-class pointer or reference.

3. OBJECTIVES

- To design a class hierarchy that realistically models a bank's Savings and Current accounts.
- To enforce encapsulation so that sensitive data such as balance cannot be modified directly from outside the class.
- To apply access specifiers (private, protected, public) appropriately to control visibility of data and methods.
- To use inheritance so that common attributes and behaviour are written once in a base class and reused by derived classes.
- To demonstrate function overloading with multiple versions of deposit().
- To demonstrate function overriding with account-specific versions of calculateInterest().
- To use virtual functions to achieve runtime polymorphism when accounts are handled through base-class pointers.
- To evaluate how these OOP principles improve data security, flexibility, reusability and maintainability.

4. OOP CONCEPTS USED

OOP Concept	Where It Is Used
Class & Object	BankAccount, SavingsAccount and CurrentAccount classes; sa and ca objects created in main().
Encapsulation	accountNumber, accountHolderName and balance are private/protected and accessed only through member functions.
Access Specifiers	private members in derived classes (interestRate, overdraftLimit); protected members in the base class shared with derived classes; public interface (deposit, withdraw, displayDetails).

OOP Concept	Where It Is Used
Inheritance	SavingsAccount and CurrentAccount both inherit publicly from BankAccount.
Function Overloading	Two versions of deposit(): deposit(double) and deposit(double, string).
Function Overriding	calculateInterest() and displayDetails() are redefined differently in SavingsAccount and CurrentAccount.
Virtual Function	calculateInterest() and displayDetails() are declared virtual in BankAccount, enabling runtime polymorphism through base-class pointers.
Data Security	balance can only be changed via deposit()/withdraw(), which validate the amount before changing it.
Reusability	Constructor logic, getBalance(), getAccountNumber() and the base part of displayDetails() are written once and reused by both derived classes.

PSEUDOCODE :-

START

Create a base class BankAccount

Store:

name

accountNumber

balance

Define deposit(amount)

Add amount to balance

Define deposit(amount, mode)

 Add amount to balance

 Display deposit mode

Define withdraw(amount)

 IF amount <= balance

 Subtract amount from balance

 ELSE

 Display "Insufficient balance"

 END IF

Define calculateInterest()

 Display general interest

Define displayDetails()

 Display name, account number and balance

Create SavingsAccount derived from BankAccount

 Override calculateInterest()

$\text{interest} = \text{balance} \times 5 / 100$

 Display interest

Create CurrentAccount derived from BankAccount

 Override calculateInterest()

$\text{interest} = \text{balance} \times 2 / 100$

 Display interest

MAIN

 Read customer name

 Read account number

 Read initial balance

Display:

1. Savings Account
2. Current Account

Read choice

IF choice = 1

 Create SavingsAccount object

ELSE

 Create CurrentAccount object

END IF

Read deposit amount

Call deposit()

Read withdrawal amount

Call withdraw()

Call displayDetails()

Call calculateInterest()

// Runtime polymorphism

Call deposit(amount, mode)

// Function overloading

Call displayDetails()

Delete account object

END

C++ CODE :-

```

#include <iostream>
#include <string>
using namespace std;

// Base Class
class BankAccount {
protected:
    string name;
    int accountNumber;
    double balance;

public:
    // Constructor
    BankAccount(string n, int acc, double bal) {
        name = n;
        accountNumber = acc;
        balance = bal;
    }

    // Function Overloading
    void deposit(double amount) {
        balance += amount;
        cout << "Amount deposited: Rs." << amount << endl;
    }

    void deposit(double amount, string mode) {
        balance += amount;
        cout << "Amount deposited: Rs." << amount
            << " through " << mode << endl;
    }

    // Encapsulation - controlled access
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;

```

```

        cout << "Amount withdrawn: Rs." << amount << endl;
    } else {
        cout << "Insufficient balance!" << endl;
    }
}

```

// Virtual function for Runtime Polymorphism

```

virtual void calculateInterest() {
    cout << "General bank account interest." << endl;
}

```

```

void displayDetails() {
    cout << "\n--- Account Details ---" << endl;
    cout << "Name      : " << name << endl;
    cout << "Account Number : " << accountNumber << endl;
    cout << "Balance    : Rs." << balance << endl;
}

```

```

virtual ~BankAccount() {}
};

```

// Derived Class

```

class SavingsAccount : public BankAccount {
public:

```

```

    SavingsAccount(string n, int acc, double bal)
        : BankAccount(n, acc, bal) {}

```

// Function Overriding

```

void calculateInterest() override {
    double interest = balance * 5 / 100;
    cout << "Savings Account Interest (5%): Rs."
        << interest << endl;
}
};

```

```

// Derived Class
class CurrentAccount : public BankAccount {
public:
    CurrentAccount(string n, int acc, double bal)
        : BankAccount(n, acc, bal) {}

    // Function Overriding
    void calculateInterest() override {
        double interest = balance * 2 / 100;
        cout << "Current Account Interest (2%): Rs."
            << interest << endl;
    }
};

int main() {
    string name;
    int accountNumber;
    double balance, amount;

    cout << "==== Bank Account Management System =====" << endl;

    cout << "\nEnter customer name: ";
    getline(cin, name);

    cout << "Enter account number: ";
    cin >> accountNumber;

    cout << "Enter initial balance: Rs.";
    cin >> balance;

    cout << "\nChoose Account Type:" << endl;
    cout << "1. Savings Account" << endl;
    cout << "2. Current Account" << endl;
    cout << "Enter choice: ";

```



```
int choice;
cin >> choice;

BankAccount* account;

if (choice == 1)
    account = new SavingsAccount(name, accountNumber, balance);
else
    account = new CurrentAccount(name, accountNumber, balance);

cout << "\nEnter deposit amount: Rs.";
cin >> amount;

account->deposit(amount);

cout << "\nEnter withdrawal amount: Rs.";
cin >> amount;

account->withdraw(amount);

account->displayDetails();

// Runtime Polymorphism
cout << "\n--- Interest Calculation ---" << endl;
account->calculateInterest();

// Function Overloading
cout << "\n--- Function Overloading Example ---" << endl;
account->deposit(500, "Online Transfer");

account->displayDetails();

delete account;

cout << "\nProgram completed successfully." << endl;
```

```
    return 0;
}
```

OUTPUT :-

```
C:\Users\Jagadeesh\OneDrive\Documents\bankk.exe
Enter deposit amount: Rs.2000
Amount deposited: Rs.2000

Enter withdrawal amount: Rs.1000
Amount withdrawn: Rs.1000

--- Account Details ---
Name       : Naveen
Account Number : 1001
Balance    : Rs.11000

--- Interest Calculation ---
Savings Account Interest (5%): Rs.550

--- Function Overloading Example ---
Amount deposited: Rs.500 through Online Transfer

--- Account Details ---
Name       : Naveen
Account Number : 1001
Balance    : Rs.11500

Program completed successfully.

-----
Process exited after 86.06 seconds with return value 0
Press any key to continue . . . █
```

10. ANALYSIS AND EVALUATION

10.1 Classes, Objects and Abstraction

Modelling `BankAccount`, `SavingsAccount` and `CurrentAccount` as classes allowed the program to represent real bank accounts as self-contained objects that bundle data with the operations that are valid on that data. This is a direct application of abstraction: code that uses an account object only needs to know what it can do (deposit, withdraw, calculate interest) and not how balance is stored or validated internally.

10.2 Encapsulation and Access Specifiers

Declaring `balance` as protected (rather than public) inside `BankAccount`, and `interestRate` / `overdraftLimit` as private inside the derived classes, ensures that no code outside the class hierarchy can assign an arbitrary value to these fields directly. All modification is funnelled through `deposit()` and `withdraw()`, which validate that amounts are positive and that withdrawals do not exceed what is allowed. This evaluation confirms that access specifiers are not just syntax — they are what make the data-security guarantee enforceable.

10.3 Inheritance and Reusability

Because `SavingsAccount` and `CurrentAccount` inherit from `BankAccount`, the constructor logic, `getBalance()`, `getAccountNumber()`, the default `withdraw()` and the shared part of `displayDetails()` are written exactly once. Without inheritance, this logic would have to be duplicated in every account class, increasing the risk of inconsistent bugs (for example, one class validating deposits and another forgetting to).

10.4 Polymorphism: Overloading vs. Overriding

Function overloading (two `deposit()` signatures) improves flexibility at compile time — the caller simply picks the version whose parameters match their intent. Function overriding, combined with the `virtual` keyword, provides flexibility at run time: the same call, `pointer->calculateInterest()`, produces different, class-appropriate behaviour depending on the actual object type, without the caller needing to know or check that type. This is what allowed the accounts array in `main()` to treat every account uniformly while still getting correct, type-specific results — the single most powerful practical benefit of runtime polymorphism observed in this assignment.

14. CONCLUSION

This assignment successfully implemented a Bank Account Management System in C++ that demonstrates classes and objects, encapsulation, access specifiers, inheritance, function overloading, function overriding, virtual functions and runtime polymorphism, data security, and code reusability, all working together in one coherent application. The evaluation confirmed that these OOP principles directly translate into practical software-engineering benefits: safer data handling, less duplicated code, and a design that can be extended to new account types with minimal changes to existing, tested code. Overall, the exercise reinforced that Object-Oriented Programming is best learned not as a list of separate rules but as a connected set of tools for modelling real-world systems reliably and maintainably.

Code of Conduct

I **M.Naveen /192411134 & Haleez Mohammed/192511460** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

_____K.Veena devi_____

Faculty In-charge

Name:

Date:

Course Coordinator

Name:

Date:

